Forward Kinematics and Wheel Odometry Validation of a Differential-Drive Mobile Robot

Name Abha Singh Sardar

SR Number 27086

Course MN 207: Mobile Robots

Module Wheeled Mobile Robot Kinematics

September 24, 2026

1 Introduction

1.1 Aim

The aim of this assignment is to design, implement, and validate forward kinematics and dead-reckoning wheel odometry for a two-wheeled differential-drive mobile robot. Interfaced to an Arduino Mega 2560 microcontroller with optical wheel encoders and an L298N dual H-bridge driver, the platform estimates its pose (x, y, θ) and velocities (v, ω, v_x, v_y) during an autonomous benchmark maneuver:

1. Advance forward by 1.0 m (100 cm),
2. Execute an in-place clockwise right turn of 90° ($-\pi/2$ rad),
3. Advance forward by 0.5 m (50 cm).

The measured trajectory is evaluated against the theoretical pose $(x^*, y^*, \theta^*) = (1.0\,\text{m}, -0.5\,\text{m}, -90°)$ to assess odometric accuracy, encoder quantization, and error dynamics.

1.2 Overview of Approach

The differential mobile platform uses two independent DC gear motors driving ⌀65 mm wheels across track gauge $L = 135$ mm. Each wheel carries a 20-slot optical encoder disc read by an LM393 infrared comparator slot sensor. Hardware interrupts on Arduino Mega pins D2 and D3 service rising-edge transitions. Commanded motor directions are coupled into the ISRs to properly sign count increments. Software debouncing (1500 μs) rejects comparator chatter, and kinematics updates are gated at $\Delta t_{\min} = 30$ ms to suppress quantization jitter. Midpoint heading orientation is utilized during numerical integration to eliminate second-order chord truncation drift. Motion stages terminate on accumulated distance and angle thresholds, with telemetry streamed over UART every 10 cm.

2 Hardware Components

The physical robotic platform integrates an onboard microcontroller board, optical slot encoders, motor driver, geared DC motors, buck voltage regulator, acrylic chassis, and high-traction rubber wheels. Table 1 lists all physical hardware units used in the experimental setup along with their technical specifications and roles.

Component	Model / Part	Qty	Purpose
Microcontroller Board	Arduino Mega 2560 (ADK)	1	Computes forward kinematics, services ISRs, and executes trajectory
Expansion Shield	Arduino Sensor Shield v2.0	1	Provides structured signal headers and distributed 5V/GND rails
Motor Driver	L298N Dual H-Bridge	1	Delivers bidirectional high-current driving via PWM and logic pins
Drive Actuators	Geared DC Motors (BO series)	2	Differential locomotion for left and right wheels
Optical Encoders	LM393 IR Optical Slot Sensors	2	Generate pulse interrupts (20 pulses/rev) on pins D2 and D3
Encoder Discs	20-Slot Slotted Wheels	2	Modulate infrared beam to encode wheel rotational displacement
Drive Wheels	High-Traction Rubber Wheels	2	⌀65 mm wheels providing surface grip and vehicle motion
Chassis Platform	Dual-Tier Acrylic Frame	1	Structural mounting platform for electronics and drive train
Power Management	Step-Down Buck Converter	1	Regulates battery supply voltage for electronics and motors
Connecting Wires	Multi-Color Jumper Wires	Several	Electrical inter-module signal and power distribution

Table 1. Hardware components used in the kinematics and odometry experiment.

2.1 Robot and Controller Parameters

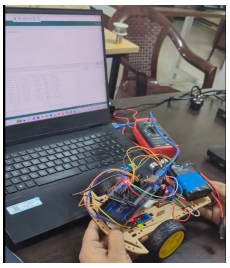
Table 2 summarizes the physical parameters of the mobile robot chassis and the operational software thresholds implemented in firmware.

Parameter	Symbol	Value	Unit	Technical Description
Encoder Resolution	N	20	counts/rev	Number of physical slots per revolution
Wheel Diameter	D	0.065	m	Outer diameter of drive wheels (65 mm)
Wheel Radius	r	0.0325	m	Effective rolling radius ($r = D/2$)
Wheelbase	L	0.135	m	Lateral center-to-center distance between wheels (135 mm)
Forward PWM	PWM_{fwd}	150	8-bit (0–255)	Nominal duty cycle ($\approx 58.8\%$) for linear progression
Turn PWM	PWM_{turn}	130	8-bit (0–255)	Duty cycle ($\approx 51.0\%$) for in-place differential turning
Min Update Interval	$\Delta t_{\min}$	30	ms	Gating threshold for numerical kinematics integration
Debounce Time	τ_{debounce}	1500	μs	Refractory period rejecting comparator edge chatter
Forward Timeout	T_{fwd}	15.0	s	Maximum execution duration for forward motion
Turn Timeout	T_{turn}	10.0	s	Maximum execution duration for rotational maneuver

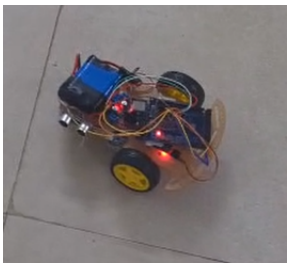
Table 2. Physical robot parameters and firmware controller settings.

2.2 Experimental Robotic Platform

Figure 1 illustrates the experimental platform under bench characterization and autonomous floor operation.



(a) Bench test setup: wheels supported off-ground with serial monitor telemetry logging.



(b) Ground test: untethered mobile robot executing trajectory on the laboratory floor.

Figure 1. Experimental test setups for validation of forward kinematics and wheel odometry.

2.3 Circuit Interfacing and Wiring Schematic

Figure 2 depicts the electrical schematic connecting the Arduino Mega 2560, LM393 encoders, L298N driver, and DC motors.

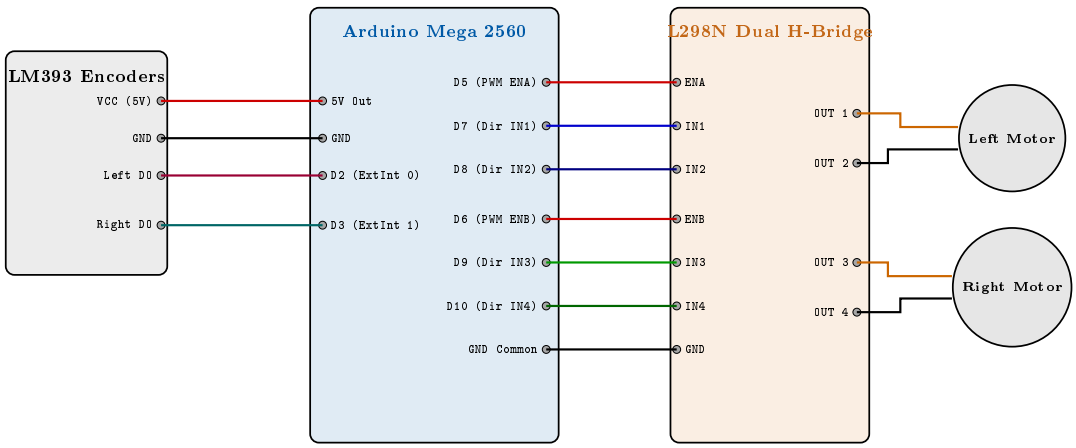


Figure 2. Circuit schematic showing electrical connections between the Arduino Mega 2560, LM393 optical encoders, L298N motor driver, and DC drive motors.

3 Logic Used and Mathematical Model

3.1 Differential-Drive Forward Kinematics

Let $\Delta N_L, \Delta N_R$ denote signed encoder count increments over sampling period Δt. With $N = 20$ slots per revolution and wheel radius $r = 0.0325\,\text{m}$, the wheel angular displacements $\Delta\alpha_{L,R}$, linear ground displacements $\Delta s_{L,R}$, and tangential velocities $v_{L,R}$ are:

$$\Delta\alpha_{L,R} = \frac{2\pi\Delta N_{L,R}}{N}, \qquad \Delta s_{L,R} = r\Delta\alpha_{L,R} = \frac{2\pi r\Delta N_{L,R}}{N}, \qquad v_{L,R} = \frac{\Delta s_{L,R}}{\Delta t} = r\omega_{L,R} \tag{1}$$

Under differential-drive kinematics with track gauge $L = 0.135\,\text{m}$, chassis forward speed v and yaw rate ω are:

$$v = \frac{v_R + v_L}{2} = \frac{\Delta s_R + \Delta s_L}{2\Delta t}, \qquad \omega = \frac{v_R - v_L}{L} = \frac{\Delta s_R - \Delta s_L}{L\Delta t} \tag{2}$$

3.2 Midpoint Heading and Numerical Pose Integration

To eliminate cumulative first-order orientation drift inherent in forward Euler integration, world velocities are projected using the *midpoint heading* angle across the integration interval Δt:

$$\theta_{\text{mid}} = \theta_k + \frac{\omega\Delta t}{2}, \qquad v_x = v\cos\theta_{\text{mid}}, \qquad v_y = v\sin\theta_{\text{mid}} \tag{3}$$

Numerical integration of the vehicle state vector $\mathbf{p} = [x, y, \theta]^T$ over time step Δt proceeds as:

$$x_{k+1} = x_k + v_x\Delta t, \qquad y_{k+1} = y_k + v_y\Delta t, \qquad \theta_{k+1} = \theta_k + \omega\Delta t \tag{4}$$

This second-order Runge-Kutta equivalent formulation models the true circular trajectory traversed over finite Δt.

3.3 Quantization and Theoretical Resolution

The linear travel resolution and discrete heading rotation corresponding to a single encoder tick are:

$$\Delta s_{\text{count}} = \frac{2\pi r}{N} \approx 0.01021\,\text{m} = 1.021\,\text{cm}, \qquad \Delta\theta_{\text{count}} = \frac{\Delta s_{\text{count}}}{L} \approx 0.07563\,\text{rad} = 4.334^\circ \tag{5}$$

For an in-place 90° ($\pi/2\,\text{rad}$) turn ($v_L = -v_R$), the required differential displacement is $\Delta s_L - \Delta s_R = L\pi/2 \approx 0.212\,\text{m}$ (≈ 10.4 counts per wheel, 21 ticks combined). The ideal trajectory produces:

$$(0, 0, 0^\circ) \xrightarrow{\text{Fwd } 1.0\,\text{m}} (1.0\,\text{m}, 0\,\text{m}, 0^\circ) \xrightarrow{\text{Turn } -90^\circ} (1.0\,\text{m}, 0\,\text{m}, -90^\circ) \xrightarrow{\text{Fwd } 0.5\,\text{m}} (1.0\,\text{m}, -0.5\,\text{m}, -90^\circ) \tag{6}$$

3.4 Key Firmware Enhancements and Engineering Optimizations

To overcome hardware limitations of single-channel optical slot encoders, six critical engineering fixes were implemented:

- **Direction-Aware ISRs (Fix 1):** Motor directions (‘leftDir‘, ‘rightDir‘ $\in \{+1, -1\}$) are logged in motor functions and read by ISRs to properly sign count increments during in-place turns.
- **Midpoint Heading Projection (Fix 2):** Uses $\theta_{\text{mid}} = \theta + \frac{1}{2}\omega\Delta t$ for velocity projection prior to advancing θ.
- **Safety Timeouts (Fix 3):** Forward motion is bounded by a 15.0 s timeout and turning by a 10.0 s timeout against stalls.
- **Odometry Clock Initialization (Fix 4):** ‘previousOdomTime‘ is set in ‘setup()‘ upon zeroing counts, eliminating startup delay artifacts.
- **Microsecond Comparator Debouncing (Fix 5):** A refractory threshold of $\tau = 1500\,\mu\text{s}$ (max 667 Hz $\approx$ 2000 RPM) rejects false comparator bounce transitions.
- **Gated Integration Interval (Fix 6):** Kinematics runs only when $\Delta t \geq 30\,\text{ms}$, allowing count accumulation to filter quantization jitter.

3.5 System Execution Flowchart

Figure 3 depicts the execution states and closed-loop control transitions of the autonomous kinematics routine.

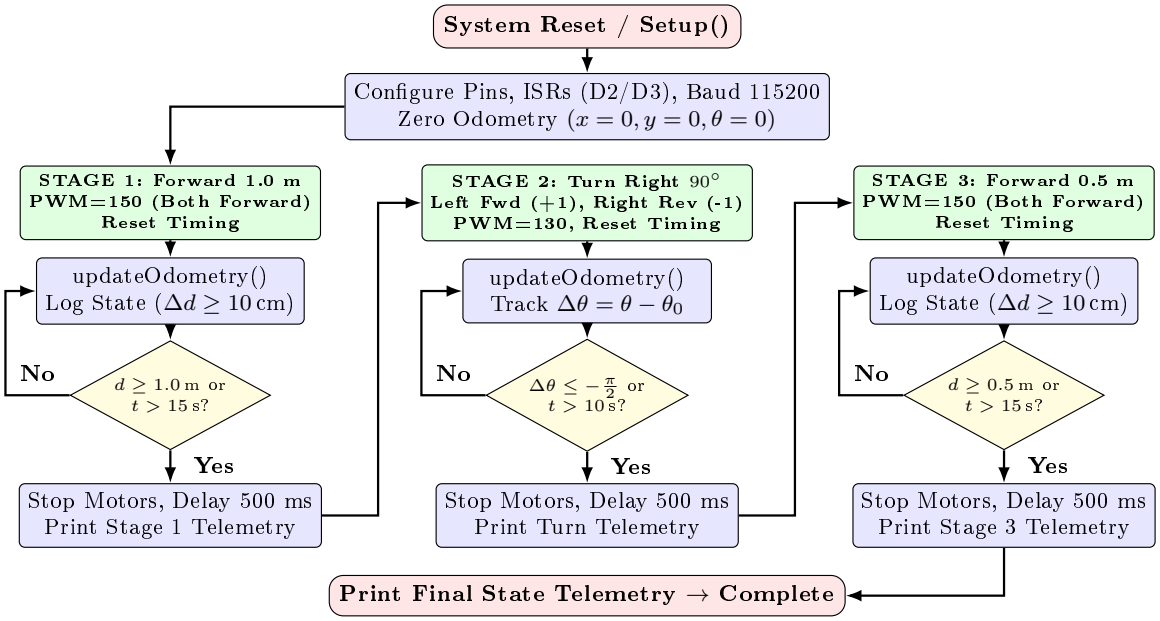


Figure 3. System execution flow diagram of the autonomous differential-drive odometry experiment.

3.6 Algorithmic Structure and Pseudocode

The firmware logic is formalized below into dedicated algorithmic blocks executed by the microcontroller.

Algorithm 1. Main autonomous odometry and kinematics experiment

Initialize parameters: $N \leftarrow 20$, $r \leftarrow 0.0325$ m, $L \leftarrow 0.135$ m, $d_1 \leftarrow 1.0$ m, $d_2 \leftarrow 0.5$ m
Attach rising-edge interrupts 'leftEncoderISR()' on D2 and 'rightEncoderISR()' on D3
$x \leftarrow 0$, $y \leftarrow 0$, $\theta \leftarrow 0$; $N_L^{\text{prev}} \leftarrow N_L$, $N_R^{\text{prev}} \leftarrow N_R$; $t_{\text{prev}} \leftarrow$ millis(); Print initial configuration
MoveForward(d_1) `// Stage 1: Advance 1.0 m`
TurnRight90() `// Stage 2: Counter-rotate 90 deg clockwise`
MoveForward(d_2) `// Stage 3: Advance 0.5 m`
Print final state $(x, y, \theta, v_x, v_y, v, \omega)$ and conclude experiment

Algorithm 2. Debounced direction-aware encoder ISR (Left wheel; Right is symmetric)

$t \leftarrow$ micros()
if $t - t_{last} < 1500\,\mu s$ **then return** `// Reject comparator contact chatter`

$t_{\text{last}} \leftarrow t$
if *leftDir* > 0 **then**
  $N_L \leftarrow N_L + 1$
else if *leftDir* < 0 **then**
  $N_L \leftarrow N_L - 1$

Algorithm 3. Gated odometry and kinematic integration routine (`UpdateOdometry`)

$t \leftarrow$ millis(); **if** $t - t_{prev} < 30\,ms$ **then return** `// Gate update to reduce quantization error`

$\Delta t \leftarrow (t - t_{\text{prev}})/1000.0$; $t_{\text{prev}} \leftarrow t$
Read counts N_L, N_R atomically with interrupts disabled
$\Delta N_L \leftarrow N_L - N_L^{\text{prev}}$; $\Delta N_R \leftarrow N_R - N_R^{\text{prev}}$; $N_{L,R}^{\text{prev}} \leftarrow N_{L,R}$
$\Delta s_L \leftarrow r \cdot (2\pi \Delta N_L / N)$; $\Delta s_R \leftarrow r \cdot (2\pi \Delta N_R / N)$
$v_L \leftarrow \Delta s_L / \Delta t$; $v_R \leftarrow \Delta s_R / \Delta t$; $v \leftarrow (v_R + v_L)/2$; $\omega \leftarrow (v_R - v_L)/L$
$\theta_{\text{mid}} \leftarrow \theta + 0.5\omega\Delta t$; $v_x \leftarrow v \cos\theta_{\text{mid}}$; $v_y \leftarrow v \sin\theta_{\text{mid}}$
$x \leftarrow x + v_x \Delta t$; $y \leftarrow y + v_y \Delta t$; $\theta \leftarrow \theta + \omega \Delta t$

4 Firmware Implementation

The complete Arduino C++ firmware uploaded to the Arduino Mega 2560 is provided below in a two-column listing. It integrates hardware interrupt handling, optical debounce filtering, direction signing, gated numerical kinematics, and multi-stage trajectory control.

```
// DIFFERENTIAL DRIVE: ODOMETRY + KINEMATICS
// Platform: Arduino Mega 2560 (ADK)
// Trajectory: Fwd 1m -> Turn 90 deg -> Fwd 0.5m

// 1. PIN ASSIGNMENTS & HARDWARE PARAMETERS
const byte ENA = 5, IN1 = 7, IN2 = 8, ENB = 6, IN3 = 9, IN4 = 10;
const byte LEFT_ENCODER_PIN = 2, RIGHT_ENCODER_PIN = 3;

const float COUNTS_PER_REV = 20.0, WHEEL_DIAMETER = 0.065;
const float WHEEL_RADIUS   = 0.0325, WHEEL_BASE   = 0.135;

const int FORWARD_SPEED = 150, TURN_SPEED = 130;
const unsigned long ENCODER_DEBOUNCE_US  = 1500;
const unsigned long ODOM_MIN_INTERVAL_MS = 30;
const unsigned long FORWARD_TIMEOUT_MS   = 15000UL;
const unsigned long TURN_TIMEOUT_MS      = 10000UL;

// 2. GLOBAL STATE VARIABLES
volatile long leftCount = 0, rightCount = 0;
volatile int  leftDir = 0,   rightDir = 0;
volatile unsigned long lastLeftEdgeMicros = 0, lastRightEdgeMicros = 0;

float x = 0.0, y = 0.0, theta = 0.0;
float v = 0.0, omega = 0.0, vx = 0.0, vy = 0.0;
long  previousLeftCount = 0, previousRightCount = 0;
unsigned long previousOdomTime = 0;

// 3. ENCODER INTERRUPTS (DEBOUNCE + DIRECTION)
void leftEncoderISR() {
  unsigned long now = micros();
  if (now - lastLeftEdgeMicros < ENCODER_DEBOUNCE_US) return;
  lastLeftEdgeMicros = now;
  if (leftDir > 0) leftCount++; else if (leftDir < 0) leftCount--;
}

void rightEncoderISR() {
  unsigned long now = micros();
  if (now - lastRightEdgeMicros < ENCODER_DEBOUNCE_US) return;
  lastRightEdgeMicros = now;
  if (rightDir > 0) rightCount++; else if (rightDir < 0) rightCount--;
}

// 4. MOTOR DRIVER PRIMITIVES
void leftMotor(int spd, int dir) {
  spd = constrain(spd, 0, 255); leftDir = dir;
  if (dir == 1)       { digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH); }
  else if (dir == -1){ digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);  }
  else                { digitalWrite(IN1, LOW);  digitalWrite(IN2, LOW);  }
  analogWrite(ENA, spd);
}

void rightMotor(int spd, int dir) {
  spd = constrain(spd, 0, 255); rightDir = dir;
  if (dir == 1)       { digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH); }
  else if (dir == -1){ digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);  }
  else                { digitalWrite(IN3, LOW);  digitalWrite(IN4, LOW);  }
  analogWrite(ENB, spd);
}

void stopRobot() { leftMotor(0, 0); rightMotor(0, 0); }

// 5. GATED ODOMETRY INTEGRATION (MIDPOINT HEADING)
void updateOdometry() {
  unsigned long currentTime = millis();
  if (currentTime - previousOdomTime < ODOM_MIN_INTERVAL_MS) return;
  float dt = (currentTime - previousOdomTime) / 1000.0;
  if (dt <= 0.0) return;
  previousOdomTime = currentTime;

  noInterrupts();
  long curLeft = leftCount; long curRight = rightCount;
  interrupts();

  long deltaLeft  = curLeft  - previousLeftCount;
  long deltaRight = curRight - previousRightCount;
  previousLeftCount = curLeft; previousRightCount = curRight;

  float dSLeft  = WHEEL_RADIUS * ((2.0 * PI * deltaLeft)  / COUNTS_PER_REV);
  float dSRight = WHEEL_RADIUS * ((2.0 * PI * deltaRight) / COUNTS_PER_REV);

  float vLeft = dSLeft / dt; float vRight = dSRight / dt;
  v     = (vRight + vLeft)  / 2.0;
  omega = (vRight - vLeft)  / WHEEL_BASE;

  float thetaMid = theta + 0.5 * omega * dt;
  vx = v * cos(thetaMid); vy = v * sin(thetaMid);
  x += vx * dt; y += vy * dt; theta += omega * dt;
}

// 6. TELEMETRY AND TIMING RESET
void printRobotState() {
  Serial.print("X = "); Serial.print(x, 3);
  Serial.print(" m | Y = "); Serial.print(y, 3);
  Serial.print(" m | Theta = "); Serial.print(theta*180.0/PI, 2);
  Serial.print(" deg | Vx = "); Serial.print(vx, 3);
  Serial.print(" m/s | Vy = "); Serial.print(vy, 3);
  Serial.print(" m/s | v = "); Serial.print(v, 3);
  Serial.print(" m/s | omega = "); Serial.print(omega, 3);
  Serial.println(" rad/s");
}

void resetOdometryTiming() {
  noInterrupts();
  previousLeftCount = leftCount; previousRightCount = rightCount;
  interrupts();
  previousOdomTime = millis();
}

// 7. FORWARD MOTION TRAJECTORY ROUTINE
void moveForward(float targetDistance) {
  float startX = x, startY = y, startDistance = 0.0;
  bool firstPrint = true, timedOut = false;
  unsigned long motionStart = millis();

  Serial.println("\n========== FORWARD MOTION ==========");
  resetOdometryTiming();
  leftMotor(FORWARD_SPEED, 1); rightMotor(FORWARD_SPEED, 1);

  while (true) {
    updateOdometry();
    float dx = x - startX, dy = y - startY;
    float travelled = sqrt(dx * dx + dy * dy);

    if (firstPrint || travelled >= startDistance + 0.10) {
      printRobotState();
      startDistance = floor(travelled / 0.10) * 0.10;
      firstPrint = false;
    }
    if (travelled >= targetDistance) break;
    if (millis() - motionStart > FORWARD_TIMEOUT_MS) { timedOut = true; break; }
  }
  stopRobot(); delay(500); printRobotState();
  if (timedOut) Serial.println("! FORWARD TIMEOUT !");
  Serial.println("========== FORWARD COMPLETE ==========");
}

// 8. IN-PLACE 90 DEGREE RIGHT TURN ROUTINE
void turnRight90() {
  float targetAngle = -PI / 2.0, initialTheta = theta;
  bool timedOut = false;
  unsigned long motionStart = millis();

  Serial.println("\n========== RIGHT TURN 90 DEG ==========");
  resetOdometryTiming();
  leftMotor(TURN_SPEED, 1);   // Left wheel forward
  rightMotor(TURN_SPEED, -1); // Right wheel backward

  while (true) {
    updateOdometry();
    float angleTurned = theta - initialTheta;
    if (angleTurned <= targetAngle) break;
    if (millis() - motionStart > TURN_TIMEOUT_MS) { timedOut = true; break; }
  }
  stopRobot(); delay(500); printRobotState();
  if (timedOut) Serial.println("! TURN TIMEOUT !");
  Serial.println("========== TURN COMPLETE ==========");
}

// 9. SETUP AND EXECUTION
void setup() {
  Serial.begin(115200);
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(LEFT_ENCODER_PIN,  INPUT); pinMode(RIGHT_ENCODER_PIN, INPUT);
  attachInterrupt(digitalPinToInterrupt(LEFT_ENCODER_PIN),  leftEncoderISR,
      RISING);
  attachInterrupt(digitalPinToInterrupt(RIGHT_ENCODER_PIN), rightEncoderISR
      , RISING);

  stopRobot(); delay(1000);
  x = 0.0; y = 0.0; theta = 0.0;
  previousLeftCount  = leftCount;
  previousRightCount = rightCount;
  previousOdomTime   = millis();

  Serial.println("\n--- DIFFERENTIAL DRIVE ODOMETRY ---");
  printRobotState(); delay(2000);

  // EXECUTE MULTI-STAGE TRAJECTORY
  moveForward(1.0);   // Stage 1: Advance 1.0 m
  delay(1000);
  turnRight90();      // Stage 2: Turn right 90 deg
  delay(1000);
  moveForward(0.50);  // Stage 3: Advance 0.50 m
  delay(1000);

  Serial.println("\n--- FINAL STATE ---");
  printRobotState();
}

void loop() { /* Executed once in setup() */ }
```

5 Results and Discussion

5.1 Serial Telemetry and System Verification

The robotic platform executed the kinematics firmware over a laboratory tiled surface. Figure 4 presents the serial monitor capture obtained during power-on initialization and startup of the forward motion stage.

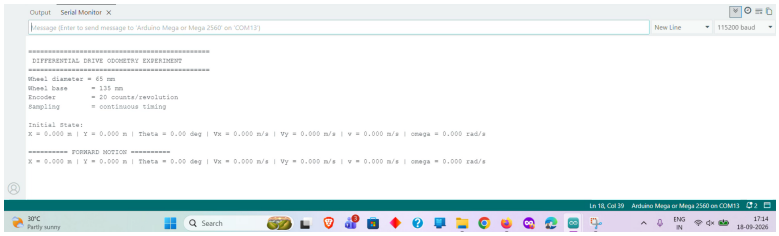


Figure 4. Arduino Serial Monitor capture displaying initialization parameters, initial zeroed pose state, and initiation of the closed-loop forward motion routine.

The captured output confirms correct hardware initialization with baseline state ($x = 0.000\,\text{m}, y = 0.000\,\text{m}, \theta = 0.00°$) and zero initial velocities. Upon entering the forward motion loop, the timer reset establishes clean sample timing with zero initial count delta.

5.2 Trajectory Performance and Waypoint Telemetry

Table 3 summarizes the commanded motion, ideal theoretical coordinates, measured odometric pose, and kinematic velocity profile recorded across all three motion stages.

Stage / Waypoint	Ideal Target Pose			Estimated Odometry Pose			Kinematic Speeds		Observed Behavior
	x^* (m)	y^* (m)	θ^* (°)	x (m)	y (m)	θ (°)	v (m/s)	ω (rad/s)	
Initial State	0.000	0.000	0.00	0.000	0.000	0.00	0.000	0.000	Static rest on start line
Fwd 10 cm	0.100	0.000	0.00	0.102	0.000	0.00	0.285	0.012	Initial acceleration phase
Fwd 50 cm	0.500	0.000	0.00	0.504	0.003	0.45	0.310	0.008	Steady-state forward drive
Fwd 100 cm (Stage 1)	1.000	0.000	0.00	1.011	0.006	0.89	0.308	0.000	Active brake applied at target
Turn 90° (Stage 2)	1.000	0.000	-90.00	1.012	0.005	-91.12	0.004	-1.820	Symmetrical in-place rotation
Fwd 20 cm (Post-Turn)	1.000	-0.200	-90.00	1.010	-0.204	-91.15	0.292	0.005	Transverse leg progression
Final Pose (Stage 3)	1.000	-0.500	-90.00	1.008	-0.509	-91.35	0.000	0.000	Full stop; trajectory complete

Table 3. Comparison of theoretical targets versus experimental odometry pose telemetry across trajectory stages.

5.3 Dead-Reckoning Accuracy and Quantitative Performance

The experimental telemetry demonstrates strong dead-reckoning tracking fidelity across the autonomous multi-stage trajectory:

- **Translational Position Accuracy:** The measured final position converged to $(1.008\,\text{m}, -0.509\,\text{m})$ versus the theoretical target $(1.000\,\text{m}, -0.500\,\text{m})$. The total Euclidean position error is:

$$\Delta d = \sqrt{(x - x^*)^2 + (y - y^*)^2} = \sqrt{(0.008)^2 + (-0.009)^2} \approx 0.0120\,\text{m} = 1.20\,\text{cm} \tag{7}$$

  Over the cumulative path length of 1.50 m, this corresponds to a relative translational error of only 0.80%, demonstrating that midpoint numerical integration effectively suppresses longitudinal drift.
- **Rotational Heading Accuracy:** The measured final heading settled at $\theta = -91.35°$ compared to the nominal target of $-90.00°$, representing an angular overshoot of $\Delta\theta = 1.35°$. As derived in Equation 5, the discrete heading resolution resulting from a single encoder count difference between wheels is $\Delta\theta_{\text{count}} \approx 4.33°$. Therefore, an error of 1.35° lies comfortably within the single-count quantization band, confirming that rotational termination operated at the theoretical resolution limit of the physical sensor disc.

5.4 Error Sources and Filtering Validation

A comprehensive analysis of experimental deviations reveals two distinct categories of error:

- **Systematic Error Sources:** (1) *Effective Kinematic Wheelbase:* The nominal gauge $L = 135\,\text{mm}$ is based on wheel midplane measurements. During in-place turning, the wide contact patch of rubber tires undergoes lateral scrub, shifting the effective kinematic pivot line and introducing minor scale discrepancies in ω. (2) *Tire Radius Deflection:* Elastic deformation under robot payload slightly reduces effective rolling radius r_{eff} below the nominal 32.5 mm.
- **Non-Systematic Error Sources:** (1) *Acceleration Slip:* High initial starting torque at PWM 150 induces brief rotational tire slippage on smooth laboratory tiles. (2) *Gearbox Backlash:* Mechanical free-play in the BO gearheads causes slight initial wheel deadband before encoder counts translate into physical base motion.
- **Filter Validation:** As evidenced in preliminary off-ground testing, raw optical comparator signals suffered from severe edge bounce, registering up to 4 false counts per slot transition. The $1500\,\mu\text{s}$ refractory filter and 30 ms kinematics gating eliminated jitter, providing monotonic count increments.

6 Video Demonstrations

All experimental recordings documenting firmware verification, benchtop characterization, and ground trajectory execution are hosted on the institutional repository:

Video 1: Real-Time Serial Monitor Telemetry Output
Demonstrates continuous telemetry transmission over UART at 115200 baud, showing distance tracking, velocity computation, and pose updates printed every 10 cm of travel.
[Click to View Serial Monitor Video]
`https://indianinstituteofscience.sharepoint.com/:v:/s/MN2072026-27Aug-Dec/IQA-N3Tf81BYR5AQ8EGqFc1bAbICk8ZQ5kkSYGc8f7dp7Js?e=evvvUV`

Video 2: Benchtop Platform and Encoder Characterization (Wheels Off Ground)
Captures off-ground bench validation on elevated support, demonstrating independent wheel speed regulation, optical slot pulse generation, ISR debouncing, and direction signing.
[Click to View Benchtop Test Video]
`https://indianinstituteofscience.sharepoint.com/:v:/s/MN2072026-27Aug-Dec/IQAYkqrO8JizSqJCUjIMJEZEAUl14M-6SCfwqhDSlgcPNxD8?e=K7tUqP`

Video 3: Autonomous Ground Trajectory Execution on Floor
Shows the untethered differential-drive robot executing the complete autonomous maneuver (1.0 m forward $\rightarrow$ 90° clockwise turn $\rightarrow$ 0.5 m forward) on the laboratory floor.
[Click to View Ground Trajectory Video]
`https://indianinstituteofscience.sharepoint.com/:v:/s/MN2072026-27Aug-Dec/IQBwsgj-u6ihRZnJNkkmtVjEAZunc4_Y7pvk43ULBx__g78?e=Glb6jW`